

Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Arquitectura de computadoras
Practico N°1: ALU

Autores
Delamer Ignacio
Rivarola A. Ignacio

Introducción

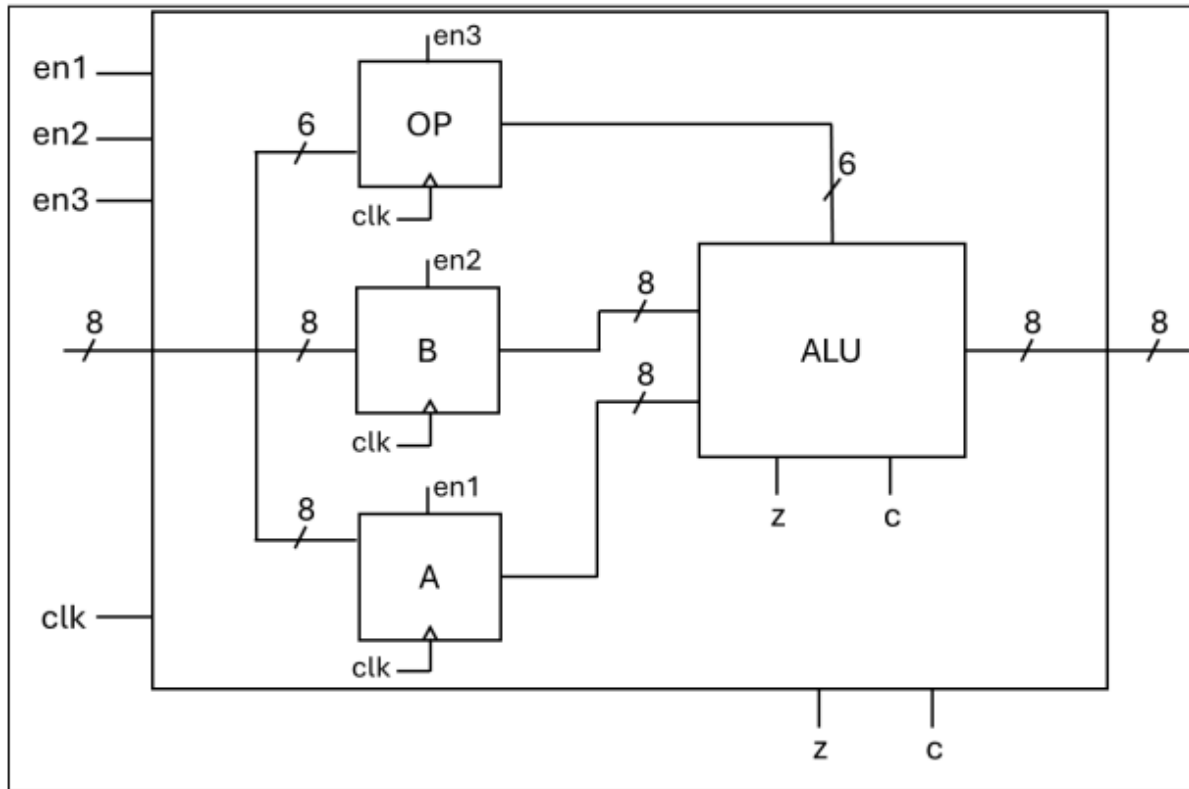
El objetivo de este trabajo es concebir, implementar y validar una **Unidad Aritmético-Lógica (ALU)** parametrizable que opere sobre datos binarios de **8 bits**. La ALU es un bloque central de cualquier arquitectura digital, responsable de ejecutar cálculos aritméticos y operaciones lógicas en procesadores, microcontroladores y sistemas embebidos.

Para alcanzar dicho objetivo, se adoptó una **arquitectura por módulos** que separa la captura de operandos, el registro del código de operación y la lógica de cómputo. Esta descomposición mejora la **trazabilidad**, **reutilización** y **verificación** del diseño. El sistema incorpora **reloj**, **reset sincrónico** y **señales de habilitación**, de modo que el flujo de datos se modele de forma secuencial y controlada.

La verificación se llevó a cabo mediante un **testbench automatizado** que genera combinaciones aleatorias de entradas, compara contra un modelo de referencia y reporta cualquier discrepancia. Las funciones implementadas abarcan **suma**, **resta**, **AND**, **OR**, **XOR**, **NOR** y **desplazamientos** (lógico y aritmético). Este enfoque permite validar el comportamiento y fundamentar las decisiones de diseño (parametrización, manejo de flags, etc.).

Desarrollo

Enfoque modular



El diagrama de bloques organiza el sistema en tres registros de entrada (**A**, **B** y **OP**) que comparten una línea de **datos** y se actualizan de forma independiente mediante señales de **enable** (**en1**, **en2** y **en3**), sincronizadas por clock (**clk**). Los operandos (**A** y **B**) alimentan la ALU con buses de 8 bits, mientras que **OP** provee un **opcode** de 6 bits que selecciona la operación. La ALU produce **result** (8 bits) y los flags **z** (cero) y **c** (acarreo/préstamo).

Bloques

Bloques A y B – Registros de operandos. Incluyen reloj, habilitación y la entrada **data** compartida. Su salida de 8 bits está conectada directamente a la ALU. Estos registros emulan el comportamiento de un datapath real, permitiendo cargar cada operando en ciclos distintos.

Bloque OP – Registro de operación. Al igual que los bloques A y B, cuenta con reloj y habilitación propia, toma los **6 LSB** de **data** y los expone como **opcode**. De esta forma, se elige la operación de la ALU (p. ej., ADD, SUB, AND...).

Bloque ALU – Núcleo de procesamiento. Recibe **A**, **B** y **OP** y entrega **result**, **z** y **c**. La lógica interna selecciona la operación correspondiente y calcula los modifica los flags de status.

Ventajas

Tomar este enfoque modular, nos brinda ciertas ventajas como la **reutilización**, ya que cada bloque puede insertarse en otros diseños sin realizar cambios. También es un diseño **Escalable**, permitiendo ampliar las operaciones y anchos de datos, sin tener que realizar un rediseño. Su **Claridad estructural** simplifica la lectura y la simulación. Y por ultimo, este diseño tambien nos permite modificar la **Parametrización**, es decir, nos permite modificar el ancho de datos (N_data) y del opcode (N_op), permitiendo ser ajustados para 8, 16, 32 bits o incluso mas.

Módulos

top_module: Integración

Integra los registros y la ALU. Recibe una única **data** y las señales de habilitación para cargar **A**, **B** y **OP** en ciclos diferentes, emulando un datapath secuencial. También centraliza la **lógica de control** y facilita la simulación por escenarios.

data_module: Registro sincronizado

Es un registro con **clk**, **reset síncronico** y **enable**, reutilizado para los datos **A** y **B**, y también para **OP**. Su parametrización mediante la variable **N** permite instanciarlo con distintos anchos de datos. Cabe aclarar que se utilizaron buenas prácticas como la del disparo por **posedge clk**, reset síncrono y encapsulamiento del dato.

alu: Unidad Aritmético-Lógica

Implementa operaciones **aritméticas** (ADD, SUB), **lógicas** (AND, OR, XOR, NOR) y **desplazamientos** (SRL, SRA). A su vez, se tiene en cuenta el **carry** y el **zero**. En suma/resta se emplea concatenación {carry, S} para capturar el bit extra. El **desplazamiento aritmético** utiliza ">>>" con operandos signed para preservar el signo. La cantidad de bits de desplazamiento se gestiona con la funcion \$clog2(N_data), la cual devuelve el mínimo número entero de bits necesario para representar un rango de valores hasta N_data - 1.

Complemento a dos. La ALU interpreta los datos como signed, habilitando representación de negativos en complemento a dos y manteniendo consistencia en comparaciones y desplazamientos.

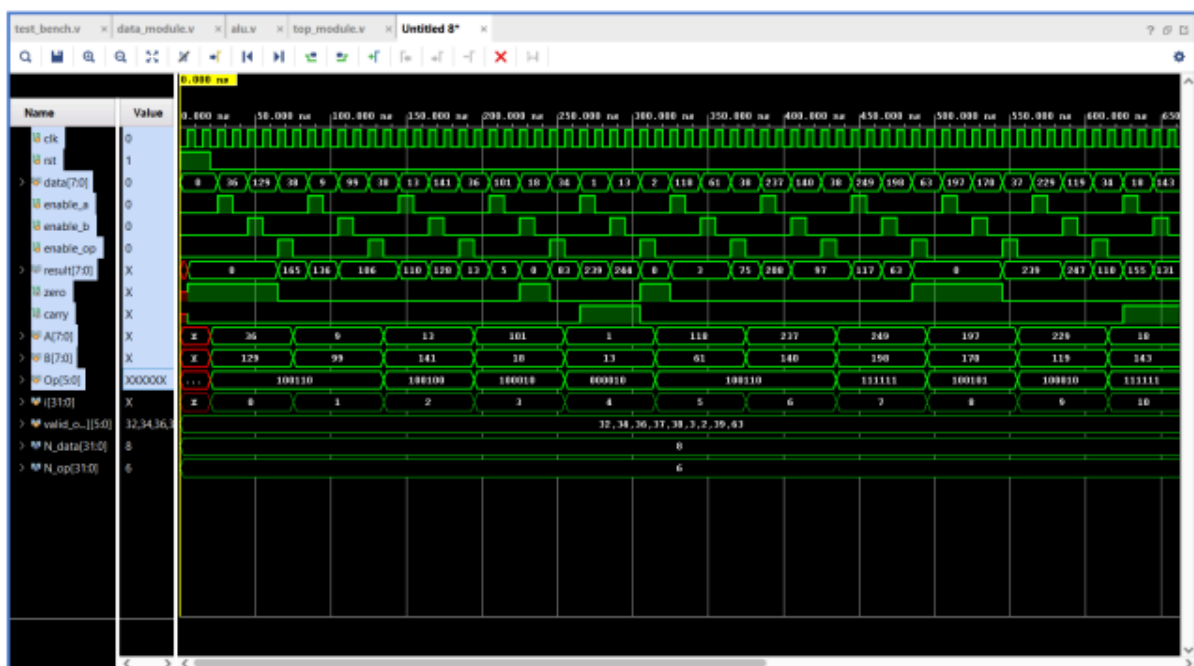
Testbench

Se desarrolló un **banco de pruebas** que ejecuta 100 iteraciones con **operandos aleatorios** y selección válida / inválida de **opcode**. Mediante **enable_a**, **enable_b** y **enable_op** se reproduce la carga secuencial de registros. Una **task** `check_result` calcula el valor de referencia y contrasta **result**, **carry** y **zero** con la salida de la ALU. En caso de que haya un error, emite un mensaje detallado y finaliza con `$fatal`.

Resultados

Resultados de la simulación

HACER NUESTRO PROPIO SCREEN COMO ESTE



Aquí podemos ver la evolución de **clk**, **reset**, señales de **enable**, entradas **dataA/dataB** y salidas (**result**, etc.). Se aprecia también, la actualización de registros en flanco positivo de **clk** cuando **enable** está activo, verificando el control secuencial del diseño.

Chequeo por consola

PONER SCREEN DE LA CONSOLA

La salida del simulador reporta las operaciones ejecutadas y los flags. Casos como $0001 + 0001 = 0002$ con `carry = 0` confirman la consistencia entre el **modelo de referencia** y la ALU.

Conclusión

El desarrollo permitió **conectar teoría y práctica** en el diseño de una ALU parametrizable. Se implementaron y validaron operaciones aritméticas y lógicas, se comprendió el rol de los **flags** y se practicó la representación en **complemento a dos** para operandos `signed`. El **testbench aleatorio** aceleró la detección de errores y aportó **trazabilidad**. La **modularidad** logró un diseño limpio, escalable y fácil de defender.